Operating Systems — Assignment 8

Virtual Memory, Address Translation, and Demand Paging

Student: Srivalli Gedela **Roll number:** 2401MC38

Submission folder: 2401mc38_srivalli_asgn_08

1. Assignment overview

This assignment explores virtual-memory mechanisms in xv6 through four exercises: translating user virtual addresses, reporting physical-memory usage, inspecting page tables, and implementing lazy allocation. The notes below summarize the implementation and observations documented in the supplied assignment materials; confirm the commands and measurements against the version you ran before submission.

Q1 — Virtual-to-physical address translation (va2pa)

The system call looks up a user virtual address in the current process page table. The page-frame address is obtained by masking off page-entry flags, then the page offset is combined with it. A mapping is accepted only when the entry is present and user-accessible; invalid or inaccessible mappings return an error value.

Q2 — Physical-memory statistics (getmeminfo)

The memory-information interface reports total and free physical pages along with process memory and page-table overhead. Allocation and release counters are updated in the allocator while holding its lock, so concurrent updates do not race. Page-table accounting includes the page directory and allocated page-table pages.

Q3 — Page-table inspection (vmprint)

The page-table printer traverses user mappings and reports their virtual/physical mapping and permission flags. Kernel-space entries are handled separately, and the guard page can be recognized as present but lacking the user-accessible permission.

Q4 — Lazy allocation / demand paging

Positive heap growth updates the process size without immediately allocating every physical page. When a valid address in the expanded heap is accessed, the page-fault handler allocates and clears a frame, then maps it with user and write permissions. Fork handling must tolerate intentionally unmapped pages in the lazy range.

2. Issues and resolutions recorded

Issue	Resolution described in the supplied notes
Address contained PTE flags	Mask the page-entry flags before combining the physical page address with the offset.
Memory counter mismatch	Protect allocator counter changes with the allocator lock.
Fork failed on lazy pages	Skip page-table entries that are not present rather than treating them as fatal.
Kernel syscall touched an unallocated user buffer	Handle valid lazy-heap page faults during the relevant kernel access path.

3. Test observations

The following values were present in the report included in the uploaded archive. They are reproduced as reference observations, not independently rerun measurements. Verify them against the screenshots and your own xv6 run.

Q2 — Memory statistics

Phase	Free pages	User pages	Recorded change
Initial state	56,750	3	Baseline
After sbrk(+16 KB)	56,746	7	–4 free pages
Child after fork	56,737	7	–9 free pages
After child reaped	56,746	7	+9 free pages restored

Q4 — Lazy allocation

Phase	Free pages	User pages	Mapped frames	Recorded observation
Initial state	56,750	3	3	Baseline
After sbrk(32 MB)	56,750	8,195	3	No frames allocated for the expanded range
Touch 3 pages	56,747	8,195	6	Three frames allocated on access
Syscall buffer read	56,746	8,195	7	One more frame allocated
Child reaped	56,746	8,195	7	Mapped child frames reclaimed

4. Screenshot index

The `screen shots/` directory contains one image per question: `q1_ss.png`, `q2_ss.png`, `q3_ss.png`, and `q4_ss.png`. Match each image to the corresponding test output when reviewing the final submission.

Review note: This report was assembled from the provided group-assignment materials and archive. Ensure the author details, implementation descriptions, and test measurements accurately reflect your own submission and course requirements.